



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC2133 – Estructuras de Datos y Algoritmos

2025 - 2

Tarea 1

Fecha de entrega código: 29 de septiembre, 23:59 Chile continental.

Link a generador de repos: [CLICK AQUÍ](#)

Objetivos

- Implementar y comparar algoritmos de ordenamiento.
- Seleccionar un algoritmo de ordenamiento óptimo según el caso.
- Diseñar e implementar ABB para resolver problemas de búsqueda.

Introducción



Tu trabajo como malévolo programador con sueldo mínimo continúa, ahora, es tiempo de pasar a la acción y comenzar a construir las herramientas que le permitirán a Gru conquistar el mundo. Sin embargo, los minions son muy revoltosos, por lo que es difícil hallarlos para asignarles sus distintas responsabilidades. Aunque sí realizan su trabajo, como son demasiados, es difícil asesorarlos y saber qué cosas que útiles han hecho y cuáles no. Es por esto que tu siguiente objetivo es automatizar la organización de minions y diseñar estructuras que permitan buscar eficientemente dentro de la casa de Gru.

Parte 1: ¡Organiza la casa de Gru!



Problema

Gru llegó a su casa y se encontró con una gran sorpresa: un enorme caos de minions corriendo de un lado a otro sin parar. Para lidiar con este problema, Gru decidió acudir a ti, su mejor trabajador y mano derecha. Tu misión será utilizar tus habilidades de programador para organizar a los minions según distintos atributos en los sectores del hogar de Gru.

Entidades

Minion: Cada minion tiene un ID único y los siguientes atributos:

- **height:** Entero positivo.
- **weight:** Entero positivo.
- **hairs:** Entero positivo.
- **evilness:** 0 si el minion es malvado, 1 si es bueno.
- **tag:** Es otra forma de identificar a un minion. Consiste en un string con 14 caracteres. Sólo tendrá letras minúsculas y no habrán repeticiones entre minions.

Sector: Un sector tiene un ID único y una lista de minions asociada. Además, todos los sectores tendrán la misma capacidad máxima (M). Las id de los sectores van de 0 a $S-1$, siendo S la cantidad de sectores.

Eventos

JOIN {id_sector} {cantidad_entrante}

Se recibe `id {id_sector}`, un entero m que indica la cantidad entrante de minions, y luego m líneas con los datos de cada minion. Debe guardar los minions en el sector con `id {id_sector}`. El sector siempre se encontrará inicialmente vacío.

input	
1	ENTER {id_sector} {cantidad_entrante}
2	{id_1 height_1 weight_1 hairs_1 evilness_1 tag_1}
3	...
4	{id_m height_m weight_m hairs_m evilness_m tag_m}

output	
1	Entraron {cantidad_entrante} minions al sector {id_sector}.

CLASSIFY {id_sector} {atributo_orden} {orden}

Ordena el sector en su totalidad según el atributo entregado y el orden señalado, el que puede ser creciente o decreciente.

output	
1	Sector {id_sector} en orden {orden} segun {atributo}:
2	Minion {id_minion_1} - {atributo_minion_1}
3	...
4	Minion {id_minion_n} - {atributo_minion_n}

Importante: En este evento, al momento de ordenar por **tag**, se tendrán que guiar por el valor ASCII de los caracteres. Según el atributo en base el cual se ordene, la complejidad debe ser óptima aplicando algoritmos conocidos de ordenamiento. ¹.

GAUZY {id_sector} {cantidad_entrante}

Considera que el sector ya tiene l minions ordenados con anterioridad, y llega una cantidad pequeña v de minions. Deberás reordenar el sector según el atributo por el que ya esté ordenado, manteniendo el orden creciente o decreciente según sea el caso. Además, siempre se cumplirá que $v < l$ y $v \leq 10$.

input	
1	GAUZY {id_sector}
2	{id_1 height_1 weight_1 hairs_1 evilness_1 tag_1}
3	...
4	{id_v height_v weight_v hairs_v evilness_v tag_v}

output	
1	Entraron {v} minions al sector {id_sector}. Sector reordenado:
2	Minion {id_minion_1} - {atributo_minion_1}
3	...
4	Minion {id_minion_n} - {atributo_minion_n}

Importante: En este evento nunca tendrás que ordenar un sector que se encuentre previamente ordenado según **tag**. Se requiere que esta operación tenga una complejidad tiempo no mayor a $O(m)$ en el peor caso, donde m es la capacidad del sector. Además, debe tener complejidad de memoria $O(1)$.

LEAVE-PLACE {id_sector} El sector con id {id_sector} se vacía por completo.

output	
1	Todos los minions del sector {id_sector} han sido eliminados.

Consideraciones Generales

- Independiente del atributo u orden (creciente o decreciente), todos los minions malvados deben quedar siempre antes que todos los minions buenos. Es importante tener en cuenta que nunca se pedirá directamente ordenar por **evilness**, ya que siempre será un criterio de ordenamiento implícito.
- Con excepción del atributo **tag**, en caso de empate (atributo y evilness iguales) siempre quedará primero el minion con menor id (sin importar si el orden es creciente o decreciente). Por otro lado, al ordenar por **tag** se mantiene el criterio de evilness. Sin embargo, en caso de empate (tag y evilness iguales), se deberá respetar el orden original en el arreglo, es decir, quedará primero el minion que aparecía antes en la lista inicial.
- Nunca van a intentar entrar más minions de los que el sector puede recibir en ese momento, es decir, no superará el límite M .
- El evento GAUZY siempre sucederá en un sector previamente ordenado.

¹Este requerimiento al igual que todos deberá coincidir con lo escrito en el informe y tu solución en C.

Ejecución

Tu programa se debe compilar con el comando **make** y debe generar un ejecutable de nombre **minions** que se ejecuta con el siguiente comando:

```
./minions input output
```

donde **input** será un archivo con los eventos a simular y **output** el archivo donde se guardarán los resultados.

En caso de querer correr el programa para ver los leaks de memoria utilizando **valgrind** deberás utilizar el siguiente comando:

```
valgrind ./minions input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de input y output.

Input

Primero recibirás el entero **S** que indica la cantidad máxima de sectores. Luego, se entrega **M** que corresponde al máximo de minions por sector, seguido del entero **E** que corresponde al número de eventos a recibir. Finalmente recibirás todos los eventos.

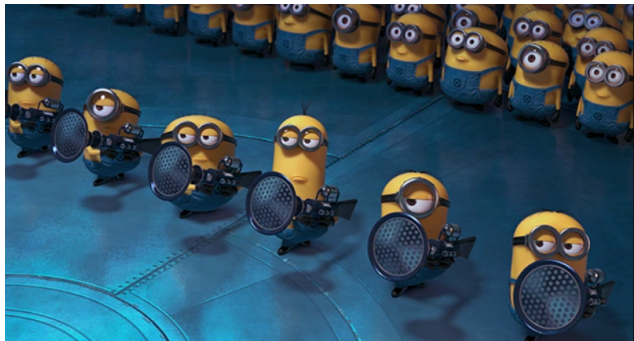
Como ejemplo tenemos el siguiente input y output:

input	
1	3
2	2
3	6
4	JOIN 1 1
5	3 2 2 2 1 nggWxImAEfUsox
6	CLASSIFY 1 hairs decreciente
7	JOIN 0 1
8	9 0 2 2 0 jenhGNrFkRiTcF
9	CLASSIFY 0 height decreciente
10	GAUZY 0 1
11	0 1 0 2 0 OMTWueGKEgMFZN
12	CLASSIFY 0 height decreciente

Output

output	
1	Entraron 1 minions al sector 1.
2	Sector 1 en orden decreciente segun hairs:
3	Minion 3 - 2
4	Entraron 1 minions al sector 0.
5	Sector 0 en orden decreciente segun height:
6	Minion 9 - 0
7	Entraron 1 minions al sector 0. Sector reordenado:
8	Minion 0 - 1
9	Minion 9 - 0
10	Sector 0 en orden decreciente segun height:
11	Minion 0 - 1
12	Minion 9 - 0

Parte 2: Armando el arma definitiva



Problema

Los minions se están preparando para construir la más rápida, la más poderosa y la más eficiente arma definitiva. Para lograr lo anterior existe una mega-fábrica que está constantemente produciendo nuevos dispositivos y chips. Para elaborar la mejor arma posible, debes encargarte de buscar eficientemente los dispositivos deseados y de gestionar correctamente los chips en base a los atributos indicados.

Entidades

Dispositivo: Cada dispositivo tiene un ID único y los siguientes atributos enteros positivos:

- power
- cost
- delay
- accuracy
- weight

Chips: Cada chip tiene un ID único y un atributo `conductivity` que es un entero positivo.

Módulo: Un módulo tiene un ID único y un conjunto de chips asociados.

La fábrica tiene múltiples dispositivos, una cantidad fija de módulos y múltiples chips asociados a algún módulo. En los eventos de esta parte debes implementar operaciones eficientes para cumplir con los requisitos de tiempo especificados.

Primero, recibirás una línea con un entero S , indicando la cantidad máxima de módulos. Luego, recibirás una línea con un entero E que indica la cantidad de eventos a recibir. A continuación, se te entregarán los E eventos, donde cada uno puede contener más de una línea de input que deberás procesar. Para la complejidad, usaremos N al referirnos a la cantidad de dispositivos, K para la cantidad de chips en un módulo y M para la cantidad de elementos encontrados en una búsqueda.

2.1 - Eventos

NEW {device_id} {power} {cost} {delay} {speed} {accuracy} {weight}

Este evento indica que la fábrica ha creado un nuevo dispositivo con las características especificadas y que debe ser ingresado al inventario.

input	
1	NEW {dev_id} {power} {cost} {delay} {speed} {accur} {weight}

El resultado esperado debe mostrar el siguiente output:

output	
1	Nuevo dispositivo con id {dev_id} registrado

Importante: Se requiere que esta operación tenga una complejidad no mayor a $\mathcal{O}(\log(N))^2$.

²Este requerimiento, al igual que todos, deberá coincidir con lo escrito en el informe y tu solución en C

SEARCH-ACCURACY {A}

Con el objetivo de crear la mejor arma, los minions necesitan buscar dispositivos con parámetros muy específicos. En este evento debes encontrar todos los dispositivos que tengan un **accuracy** igual a {A}.

En caso de que exista más de un dispositivo con el mismo **accuracy**, se deben entregar todos los que cumplan, ordenados por orden de llegada (el orden en que fueron ingresados por el evento **NEW**).

output	
1	Dispositivos con accuracy {A}:
2	dispositivo {dispositivo_id_1}
3	dispositivo {dispositivo_id_2}
4	...
5	dispositivo {dispositivo_id_M}

Importante: Se requiere que esta operación tenga una complejidad no mayor a $O(\log(N))$. No se asegura que existan dispositivos con la **accuracy** pedida, en cuyo caso se debe imprimir solo la primera línea del output.

SEARCH-SPEED-DELAY {S} {D}

Este evento busca todos los dispositivos con atributo **speed** igual a {S} y atributo **delay** igual a {D}.

Igual que en el evento anterior, en caso de existir más de un dispositivo que coincida en tener los 2 parámetros pedidos, se deben entregar todos ordenados por orden de llegada.

output	
1	Dispositivos con speed {S} y delay {D}:
2	dispositivo {dispositivo_id_1}
3	dispositivo {dispositivo_id_2}
4	...
5	dispositivo {dispositivo_id_M}

Importante: Se requiere que esta operación tenga una complejidad no mayor a $O(\log(N))$. Igual que antes, no se asegura que existan dispositivos con los parámetros pedidos.

SEARCH-COST {min} {max}

Este evento busca todos los dispositivos con atributo **cost** que esten dentro del rango [min...max], incluyendo ambos limites.

Igual al evento anterior, en caso de existir más de un dispositivo que coincida en tener los parámetros pedidos, se deben entregar todos ordenados por **cost** y si hay más de un dispositivo con el mismo **cost**, ordenados por orden de llegada.

output	
1	Dispositivos con cost en el rango {min}-{max}:
2	dispositivo {dispositivo_id_1}
3	dispositivo {dispositivo_id_2}
4	...
5	dispositivo {dispositivo_id_M}

Importante: Se requiere que esta operación tenga una complejidad no mayor a $O(\log(N) + M)$, siendo M la cantidad de dispositivos dentro del rango pedido. Igual que antes, no se asegura que existan dispositivos con los parametros pedidos.

SIMILAR-RANGE {W}

Dado un dispositivo d_1 en particular, definimos su distancia mínima $MinD(d_1)$ para el atributo `weight` como la diferencia (valor positivo siempre) más pequeña entre d_1 y algún otro dispositivo d_2 . Por ejemplo, para un conjunto de pesos: [1, 50, 21, 43, 40] la distancia mínima para cada uno de los datos es:

- $MinD(1) = 20$ (ya que $21 - 1 = 20$)
- $MinD(50) = 7$ (ya que $50 - 43 = 7$)
- $MinD(21) = 19$ (ya que $40 - 21 = 19$)
- $MinD(43) = 3$ (ya que $43 - 40 = 3$)
- $MinD(40) = 3$ (ya que $43 - 40 = 3$)

Es importante notar que para cada dato su diferencia mínima es única (**SOLO** se debe calcular la distancia con el nodo más cercano) y no es recíproca siempre (en los datos anteriores, 40 y 43 tiene diferencia mínima recíproca, pero 1 y 21 no).

Los minions son muy meticulosos, para crear su arma definitiva necesitan encontrar todos los dispositivos cuya distancia mínima bajo el atributo `weight` sea igual a W .

El resultado esperado debe mostrar el siguiente output:

output	
1	Dispositivos con distancia minima {W}:
2	dispositivo {dispositivo_id_1}
3	dispositivo {dispositivo_id_2}
4	...
5	dispositivo {dispositivo_id_M}

Importante: Se requiere que esta operación tenga una complejidad no mayor a $O(\log(N))$. Se deben imprimir todos los dispositivos que tengan la distancia mínima pedida, si hay dispositivos con distancia mínima recíproca (como el 40 y 43 en el ejemplo anterior) se deben imprimir ambos. Además, el output de este evento debe mostrar los chips encontrados ordenados por id de menor a mayor. La complejidad que esto implica no afectará a la complejidad total del evento.

BUILD {modulo_id} {K}

Este evento inicializa un módulo de id {modulo_id} con un total de {K} chips. Después, se entregan K líneas con la información de los chips asociados a este módulo.

Se recomienda investigar sobre la siguiente estructura para los eventos relacionados a chips: [Treap](#).

input	
1	BUILD {modulo_id} {K}
2	{chip_id_1} {C_1}
3	{chip_id_2} {C_2}
4	...
5	{chip_id_K} {C_K}

output	
1	Se ha creado el modulo {modulo_id} con {K} chips

Importante: Se requiere que esta operación tenga una complejidad PROMEDIO igual a $O(K \cdot \log(K))$

SEARCH-CONDUCTIVITY {modulo_id} {C}

Para este evento debes encontrar dentro del modulo {modulo_id} a todos los chips cuya **conductivity** sea igual a {C}. En caso de existir más de un chip que coincidan en **conductivity**, se deben entregar todos ordenados por orden de llegada.

El resultado esperado debe mostrar el siguiente output:

output	
1	Chips con conductivity {C} en el modulo {modulo_id}:
2	Chip {chip_id_1}
3	Chip {chip_id_2}
4	...
5	Chip {chip_id_M}

Importante: Se requiere que esta operación tenga una complejidad PROMEDIO igual a $O(\log(K))$. No se asegura que existan chips con la **conductivity** pedida.

TRANSFER {mod_1} {mod_2} {C}

Para reorganizar la distribución de chips en la fábrica, el módulo {mod_1} y el módulo {mod_2} van a separar su conjunto de chips en aquellos que tengan menor o igual **conductivity** a {C} y aquellos que tengan **conductivity** mayor a {C}. Luego, ambos modulos van a intercambiar sus conjuntos mayores. Como resultado, el módulo {mod_1} va a terminar sus chips originales menores o iguales a {C} y los chips del módulo {mod_2} mayores a {C}, y viceversa.

El resultado esperado debe mostrar el siguiente output:

output	
1	Comenzando transferencia:
2	Los chips del modulo {mod_1} se han separado en dos lotes
3	Los chips del modulo {mod_2} se han separado en dos lotes
4	Se ha transferido un lote de chips de {mod_1} a {mod_2}
5	Se ha transferido un lote de chips de {mod_2} a {mod_1}

Importante: Se requiere que esta operación tenga una complejidad PROMEDIO igual a $O(\log(K))$.

Consideraciones generales

- En todas las consultas anteriores, no se considera el tiempo de imprimir los datos en el cálculo de la complejidad, solo el de obtener los datos pedidos. Si su algoritmo encuentra los datos pedidos en $O(\log(N))$ y luego imprime en $O(M)$, siendo M la cantidad de dispositivos encontrados, entonces cumple con la complejidad de $O(\log(N))$.
- Cuando un evento de búsqueda no encuentra ningún dispositivo o chip, se debe printear sólo la primera línea del output, que indica la búsqueda realizada.

Ejecución

Tu programa se debe compilar con el comando **make** y debe generar un ejecutable de nombre **clones** que se ejecuta con el siguiente comando:

```
./factory input output
```

donde **input** será un archivo con los eventos a simular y **output** el archivo donde se guardarán los resultados.

En caso de querer correr el programa para ver los leaks de memoria utilizando valgrind, deberás utilizar el siguiente comando:

```
valgrind ./factory input output
```

Tu tarea será ejecutada con archivos de creciente dificultad, asignando puntaje a cada una de estas ejecuciones que tenga un output igual al esperado. A continuación detallaremos los archivos de input y output.

Input

El archivo de input comenzará con el entero L que corresponde al número de lugares. Luego, se entrega el entero c que indica la capacidad máxima de los lugares. Seguido, se entrega el número E de eventos y las líneas correspondientes a cada uno.

input	
1	5
2	10
3	NEW 0 8 4 6 9 2 10
4	NEW 1 10 6 2 8 5 3
5	NEW 2 5 3 1 3 5 9
6	SEARCH-ACCURACY 5
7	SEARCH-COST 2 4
8	BUILD 0 2
9	0 30
10	1 24
11	SEARCH-SPEED-DELAY 9 6
12	BUILD 1 1
13	2 30
14	NEW 7 2 3 9 2 10 1
15	SEARCH-CONDUCTIVITY 0 30

Output

output	
1	Nuevo dispositivo con id 0 registrado
2	Nuevo dispositivo con id 1 registrado
3	Nuevo dispositivo con id 2 registrado
4	Dispositivos con accuracy 5:
5	Dispositivo 1
6	Dispositivo 2
7	Dispositivos con cost en el rago 2-4:
8	Device 0
9	Device 2
10	Se ha creado el modulo 0 con 2 chips
11	Devices con speed 9 y delay 6:
12	Device 0
13	Se ha creado el modulo 1 con 1 chips
14	Nuevo dispositivo con id 3 registrado
15	Chips con conductivity 30 en el modulo 0:
16	Chip 0

Informe

Además del código de la tarea, se debe redactar un **breve** informe que explique cómo se pueden implementar algunas estructuras y algoritmos del enunciado. Les recomendamos **comenzar la tarea escribiendo este informe**, ya que pensar y armar un esquema antes de pasar al código ayuda a estructurarse al momento de programar y adelantar posibles errores que su solución pueda tener.

Este informe se debe entregar en un archivo PDF escrito usando LaTeX junto a la tarea (por lo cual tienen la misma fecha de entrega), con nombre `informe.pdf` en la raíz del repositorio. En este se debe explicar al menos los siguientes puntos:

- Parte 1
 1. Algoritmos implementados: Para el evento **CLASSIFY**, explica qué algoritmo de ordenamiento utilizaste para cada atributo y justifica su uso.
 2. Explica el algoritmo utilizado en el evento **GAUZY** y justifica su complejidad.
- Parte 2
 1. Explicación General: Menciona y explica las estructuras utilizadas y argumenta la complejidad de cada consulta.
 2. Argumenta la utilidad de usar treaps por sobre otras estructuras de datos vistas en clases para los eventos relacionados con chips.
- Bibliografía: Citar código de fuentes externas.

IMPORTANTE: Para asegurar una mejor comprensión y evaluación del informe, es **esencial** que cites el código al que haces referencia. Cada vez que menciones un fragmento de código, incluye el nombre del archivo y el número de línea correspondiente y su *hipervínculo* correspondiente. Puedes aprender cómo crear un *permalink* a tu código consultando la [documentación de GitHub](#). Esto facilitará la revisión y garantizará que el informe sea coherente con la implementación.

Ejemplo:

Si en la Parte 1 del informe estás explicando una función que implementa una determinada operación, la referencia al código podría ser así:

En nuestra implementación, utilizamos una lista enlazada para manejar la estructura de datos, como se muestra en la función `insertar_elemento` ([src/estructuras.c](#), línea 45). Esta elección permite...

De esta manera, aseguras que los revisores puedan localizar rápidamente el fragmento de código relevante y verificar que la explicación sea consistente con la implementación.

Nota: Si por alguna razón no lograste completar alguna parte del código mencionado en el informe, **aún puedes obtener puntaje** en la sección correspondiente. Para ello, debes describir claramente cuál era tu idea de implementación, los pasos que planeabas seguir, y explicar por qué no pudiste completarlo. Esto demuestra tu comprensión del problema y del proceso, lo cual también es valioso para la evaluación.

Finalmente, **puedes** referenciar código visto en clases sin necesidad de incluir el código o pseudocódigo en el informe.

Puedes encontrar un [template en Overleaf](#) disponible en este enlace para su uso en la tarea.

Para aprender a clonar un template, consulta la [documentación de Overleaf](#).

No se aceptarán informes de más de tres páginas (sin incluir bibliografía), informes ilegibles, o generados con inteligencia artificial.

Evaluación

La nota de tu tarea es calculada a partir de testcases de input/output, así como un informe escrito en LaTeX que explique un modelamiento sobre cómo abarcar el problema, las estructuras de datos a utilizar, etc. La ponderación se descompone de la siguiente forma:

Informe (20 %)	Nota entre partes (70 %)	Manejo de memoria (10 %)
10 % Parte 1	20 % Tests Parte 1	5 % Sin leaks de memoria
10 % Parte 2	50 % Tests Parte 2	5 % Sin errores de memoria

Para cada test de evaluación, tu programa deberá entregar el output correcto en **menos de 5 segundos** y utilizar menos de 1 GB de RAM³. De lo contrario, recibirás 0 puntos en ese test.

Importante: Se encuentra **estrictamente prohibido** el uso de la función `qsort()` en la tarea. Su uso implicará la **nota mínima**.

Recomendación de tus ayudantes

Estas tareas generalmente requieren de mucha dedicación, por lo que desde ya te recomendamos distribuir tu tiempo considerando los plazos definidos. Así mismo, te recomendamos fuertemente que antes de empezar a programar tu tarea, **leas el enunciado detenidamente y con anticipación para que te dediques a entender de manera profunda lo que te pedimos**. Una vez hayas comprendido el enunciado, dedica el tiempo que sea necesario para la planificación, modelación y elaboración de tu informe, para posteriormente poder programar de manera más eficiente. No olvides compilar el código como se indica en la tarea y de preguntar cualquier duda o por problemas que te surjan en [Discussions](#). Estos son consejos de tus ayudantes que te pueden ayudar a pasar el ramo.

Entrega

Código: GIT - Rama principal del repositorio asignado, que debes generar con el link dado al principio de este enunciado. Se entrega a más tardar el día de entrega a las 23:59 hora de Chile continental.

Atraso: Esta tarea considera la política de atraso y cupones especificada en el programa del curso.

Integridad académica

Este curso se adscribe al Código de Honor establecido por la Escuela de Ingeniería. Todo trabajo evaluado en este curso debe ser hecho **individualmente** por el alumno y **sin apoyo de terceros**. Se espera que los alumnos mantengan altos estándares de honestidad académica, acorde al Código de Honor de la Universidad. Cualquier acto deshonesto o fraude académico está prohibido; los alumnos que incurran en este tipo de acciones se exponen a un Procedimiento Sumario. Es responsabilidad de cada alumno conocer y respetar el documento sobre Integridad Académica publicado por la Dirección de Pregrado de la Escuela de Ingeniería.

Uso de IA

Se prohíbe el uso de herramientas de inteligencia artificial para la realización de esta tarea. Esto incluye, pero no se limita a, el uso de modelos de lenguaje como ChatGPT, Copilot, u otras tecnologías similares para la generación automática de código, soluciones, o cualquier parte del trabajo requerido. Nos reservamos el derecho de revisar todas las tareas entregadas con el fin de detectar el uso de inteligencia artificial en la creación de las soluciones. Las tareas en las que se determine que se ha utilizado IA serán consideradas como una infracción a la política de honestidad académica y serán tratadas como casos de copia.

³Puedes revisarlo con el comando `htop` u ocupando `valgrind --tool=massif`